SE6019 Security and Privacy in AI
Privacy-Preserving Inference with CKKS

Presented by

Feng Hao (G2504051D) **Wei Xiao (G2405459F)**

Nanyang Technological University

April 22, 2026

Agenda

1. Problem and Research Question
2. Method
3. Experiment and Results
4. Deep Dive
5. Discussion and Conclusion

Motivation and Scope

Motivation

In cloud inference, users send sensitive inputs (e.g., handwritten digits, medical-like signals) to a remote model. Goal: keep user inputs private while still getting useful predictions.

Project Scope

We implemented an end-to-end privacy-preserving inference pipeline:

- HE-friendly neural network (MNIST)
- CKKS encrypted inference with TenSEAL
- Quantitative comparison: plaintext vs ciphertext outputs

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

Research Question (Guide-Aligned)

RQ

Can CKKS-based encrypted inference preserve prediction correctness with acceptable numerical error, and what is the performance cost compared with plaintext inference?

Answer (from our experiments)

- Correctness: preserved on our tested set (100/100 for both plaintext and ciphertext)
- Numerical error: small (average max absolute error = 0.03123461)
- Cost: ciphertext inference is much slower (2.94 s/image vs 0.50 ms/image)

Research Question (Guide-Aligned)

RQ

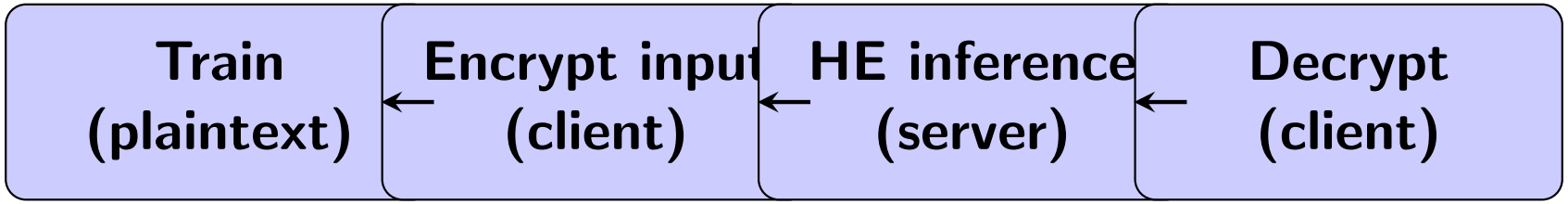
Can CKKS-based encrypted inference preserve prediction correctness with acceptable numerical error, and what is the performance cost compared with plaintext inference?

Answer (from our experiments)

- Correctness: preserved on our tested set (100/100 for both plaintext and ciphertext)
- Numerical error: small (average max absolute error = 0.03123461)
- Cost: ciphertext inference is much slower (2.94 s/image vs 0.50 ms/image)

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

System Design and Workflow



- All scripts run in Docker (reproducible environment)
- Model owner serves only encrypted computation
- Secret key stays on client side

HE-Friendly Model and CKKS Setup

Model: PolynomialNetwork

- Input: $28 \times 28 = 784$
- Layer 1: FC $784 \to 64$
- Activation: x^2 (polynomial, HE compatible)
- Layer 2: FC $64 \to 10$

CKKS Parameters

- poly_modulus_degree: 16384
- coeff_mod_bit_sizes: [60,40,40,40,60]
- global_scale: 2^{40}
- Galois + Relinearization keys enabled

Why this design?

Use shallow architecture + low multiplicative depth to keep HE computation feasible.

Key Results (Batch: 100 Samples)

Setup (for reproducibility)

MNIST test set, 3-epoch trained model, Docker execution via: `docker-he-single`, `docker-compare`, `docker-he-batch`.

Metric	Result
Plaintext accuracy	100.00%
Ciphertext accuracy	100.00%
Plaintext avg latency	0.50 ms / image
Ciphertext avg latency	2.94 s / image
Avg max absolute error	0.03123461

Interpretation

CKKS preserved classification quality in this experiment, but introduced very large latency overhead.

Single-Image Consistency Example

Observed from `make docker-he-single`

- Ground truth label: 7
- Ciphertext prediction: 7
- Server encrypted compute time: 4.97 s

Observed from `make docker-compare`

- Per-class logit differences remain small
- Largest shown absolute difference: 0.02582402 (class 9)
- Rank order remained stable for top class

Training Pipeline Details

Dataset: MNIST

- 60,000 training / 10,000 test images
- 28×28 grayscale, 10 classes
- Normalized: mean=0.1307, std=0.3081

Hyperparameters

- Optimizer: Adam ($lr = 0.001$)
- Loss: CrossEntropyLoss
- Batch size: 64
- Epochs: 3

Model Parameters

- FC1: $784 \times 64 + 64 = 50{,}240$
- FC2: $64 \times 10 + 10 = 650$
- Total: **50,890 parameters**

Design Rationale

Kept intentionally shallow: fewer parameters $\Rightarrow$ lower multiplicative depth $\Rightarrow$ feasible HE computation.

HE Inference: Step-by-Step

Three operations on ciphertext (TenSEAL)

1. `enc_out1 = enc_x.matmul(w1) + b1`
   Encrypted FC layer 1: $\mathbf{x}_{\text{enc}} \cdot W_1 + b_1$, depth $+1$
2. `enc_sq = enc_out1.square()`
   Polynomial activation x^2, depth $+1$; followed by auto-relinearization & rescaling
3. `enc_out2 = enc_sq.matmul(w2) + b2`
   Encrypted FC layer 2: produces 10 encrypted logits, depth $+1$

CKKS depth budget

- `coeff_mod_bit_sizes`: $[60, 40, 40, 40, 60]$
- 3 intermediate primes $\Rightarrow$ depth $= 3$
- Exactly fits our 3-step pipeline

Supporting keys

- Galois keys: enable ciphertext rotation for `matmul`
- Relinearization keys: shrink ciphertext after multiplication

Numerical Precision Analysis

What `compare_diff.py` measures

For the same input image, compare plain logits $\hat{y}_i$ with decrypted CKKS logits $\tilde{y}_i$:

$$\text{error}_i = |\hat{y}_i - \tilde{y}_i|, \quad i = 0, \ldots, 9$$

Observed behaviour (label = 7)

- Per-class absolute errors $\lesssim 0.031$
- Largest error $= 0.02582$ (class 9, well within CKKS noise bound)
- Rank order of all 10 logits: **identical** between plaintext and ciphertext
- Predicted class: **7** in both cases

Why approximate?

CKKS encodes real numbers with finite precision. Each multiplication introduces rounding noise $\sim 2^{-40}$ (our `global_scale`), which accumulates over 3 levels.

Latency Overhead: Why 6,000× Slower?

Operation	Plaintext	Ciphertext
FC layer (matmul)	μs	∼1–2 s
Activation (x^2)	μs	∼0.5 s
Total per image	**0.50 ms**	**2.94 s**
Slowdown	$\approx 6{,}000\times$	

Root causes

- Ciphertexts are polynomials of degree 16,384
- Each matmul requires 64 ciphertext-plaintext multiplications + ∼64 rotations
- No GPU/AVX-512 acceleration in our Docker setup (CPU only)

Mitigation directions

- CKKS batching (SIMD packing)
- Hardware acceleration (AVX-512, GPU-HE)
- Smaller `poly_modulus_degree`

Limitations and Next Steps

Current limitations

- Only tested on simple MNIST scenario
- Model is intentionally shallow (accuracy-performance tradeoff)
- Inference latency is the major bottleneck

Future work

- Test deeper HE-friendly approximations and pruning
- Explore batching/packing strategies in CKKS
- Compare CKKS with other privacy-preserving approaches (TEE/MPC)

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

Work Division

Joint: Code, Engineering & Experiments

Both members collaborated on implementing the full codebase (training, CKKS inference, batch evaluation, serialization pipeline) and running all experiments.

- **Feng Hao (G2504051D)**
  - Wrote the project report (methodology, results, analysis)
- **Wei Xiao (G2405459F)**
  - Prepared this presentation (slides, narrative, visualisations)

Final Answer to the Research Question

Answer

In our implementation, CKKS-based inference successfully protects input confidentiality and preserves prediction correctness, with small numerical approximation error. The main practical barrier is latency overhead.

Takeaway

Privacy-preserving AI inference is feasible today for low-throughput scenarios, but optimization is required for real-time production workloads.

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

References

- OpenMined, *TenSEAL*, https://github.com/OpenMined/TenSEAL
- J. H. Cheon et al., *Homomorphic Encryption for Arithmetic of Approximate Numbers (CKKS)*, ASIACRYPT 2017.
- Zama, *Concrete ML Documentation*, https://docs.zama.ai/concrete-ml